

Quick Start Guide

A. Szymczak

Program HSSC_DirectSolver

Copyright (C) 2013 E. Corona, D. Zorin, P.G. Martinsson

Contents

1	Related Papers	2
2	Algorithm Overview	3
3	HSS1D - Integral Equations on Planar Curves	5
3.1	Dependency List	5
3.2	Test Functions	5
3.3	Setup and Input	6
3.4	Sample Run	7
4	HSS2D - Integral Equations on Planar Surfaces	8
4.1	Dependency List	8
4.2	Test Functions	8
4.3	Setup and Input	9
4.4	Sample Run	9

1 Related Papers

HSSC_DirectSolver is a MATLAB implementation of [1], which is included as the file CMZ2012_paper.pdf. The six algorithms given as pseudocode in the paper correspond to the following MATLAB routines:

Algorithm 1	./HSS1D/HSS1D_invert_*.m
Algorithm 2	./HSS2D/HSS2D_invert_*.m
Algorithm 3	./HSS2D/interpolation_operator_*.m
Algorithm 4	./HSS2D/HSS2D_build_Finv_*.m
Algorithm 5	./HSS2D/HSS2D_build_E_*.m
Algorithm 6	./HSS2D/HSS2D_applyinv_*.m

The 1D solver is taken heavily from [2]. A few of the HSS1D routines are modifications of code written by P.G. Martinsson in 2008-2009.

```
./HSS1D/HSS1D_compress_*.m  
./HSS1D/HSS1D_apply_*.m  
./HSS1D/HSS1D_invert_*.m  
./HSS1D/HSS1D_applyinv_*.m  
./HSS1D/HSS1D_transforminv_*.m
```

Machine specific Fortran executables are used to calculate the Duan-Rokhlin quadrature weights for the 2D Helmholtz kernel. They are compiled from the Fortran code FORTRAN_get_DR_weights.f written by Ran Duan in August 2012 and modified by P.G. Martinsson. The quadrature rule is described in [3]. See INSTALL.txt to get an executable working on your machine.

```
./Common/FORTRAN_get_DR_weights_MAC  
./Common/FORTRAN_get_DR_weights_LINUX  
./Common/FORTRAN_get_DR_weights_WINDOWS.exe  
./Common/FORTRAN_get_DR_weights.f
```

- [1] E. Corona, D. Zorin, P.G. Martinsson. *An $O(N)$ Direct Solver for Integral Equations on the Plane*. arXiv:1303.5466.
- [2] P.G. Martinsson, V. Rokhlin. *A fast direct solver for boundary integral equations in two dimensions*. Journal of Computational Physics, 205(1), pp. 1-23, 2005.
- [3] R. Duan, V. Rokhlin. *High-Order Quadratures for the Solution of Scattering Problems in Two Dimensions*. Journal of Computational Physics, 228(6):2152-2174, 2009.

2 Algorithm Overview

Consider integral equations of the form

$$\mathcal{A}[\sigma](x) = a(x)\sigma(x) + \int_{\Omega} b(x)\mathcal{K}(\|x-y\|)c(y)\sigma(y)dy = f(x)$$

where Ω is a planar domain, a, b, c are given smooth functions, $\mathcal{K}(r)$ is related to the free-space Green's function of a PDE, and σ is unknown. We can perform a Nyström discretization to obtain the linear system $A\sigma = f$ defined on sample points $X = \{x_1, \dots, x_n\} \in \Omega$. The entries of $n \times n$ matrix A are

$$A_{i,j} = \delta_{i,j} a(x_i) + \omega_{i,j} b(x_i) \mathcal{K}(\|x_i - x_j\|) c(x_j)$$

where $\delta_{i,j}$ is the Kronecker delta and $\omega_{i,j}$ is a quadrature weight. Functions b and c often come from non-constant coefficient PDEs, but they are also useful for specifying symmetry $b \equiv c$, and translation invariance $b \equiv c \equiv 1$. This linear system will often not be fully stated. For example, we will consolidate the right hand side into a kernel function K and view matrix A as $K(p, q)$ evaluated at pairs of points.

For integral operators that arise from non-oscillatory, low-frequency elliptic PDEs, this code solves the resulting linear system in three steps: compressing A in hierarchical form (HSS), directly constructing a compressed inverse A^{-1} , and then applying the compressed inverse to f . To achieve linear complexity in 2D, an extra level of compression is needed, where certain blocks within the structure are also compressed as low-rank or HSS. When necessary, we make the distinction 'HSS1D' and 'HSS2D' for boundary and volume integral equations in the plane.

A matrix A is said to be block separable if there exist matrices $\{L_i, R_i\}$ such that each off-diagonal block $A_{i,j}$ admits the factorization $A_{i,j} = L_i M_{i,j} R_j$, where $M_{i,j}$ is smaller than $A_{i,j}$ in both dimensions. In other words, the off-diagonal blocks of A are rank-deficient. We can enforce each $M_{i,j}$ to be a submatrix of A by computing interpolative decompositions (ID) of the off-diagonal block rows and columns of A :

$$A_i^{row} = L_i A(I_i^{rsk}, :) \quad \text{and} \quad A_i^{col} = A(:, I_i^{csk}) R_i$$

where index vectors I_i^{rsk} and I_i^{csk} are the row-skeleton and column-skeleton of block i . By setting $M_{i,j} = A(I_i^{rsk}, I_j^{csk})$, we find $A_{i,j} = L_i M_{i,j} R_j$. This yields the block factorization $A = D^d + LMR$ where D, L, R are block diagonal matrices whose entries are given by $A_{i,i}, L_i, R_i$, respectively, and M is the skeleton-submatrix of A with off-diagonal blocks $M_{i,j}$.

Suppose we re-block M by merging 2x2 sets of blocks to form larger ones. A matrix is said to be *hierarchically semiseparable* (HSS) if the block-separability can be recursed on the re-blocked skeleton matrices. This leads to the *telescoping factorization* (TF)

$$A^\ell = D^\ell + L^\ell A^{\ell-1} R^\ell \quad \ell = d, \dots, 1$$

where $A = A^d$, matrices D^ℓ, L^ℓ, R^ℓ are block diagonal, and skeleton matrix $A^{\ell-1}$ is a submatrix of A^ℓ . It is useful to see the factorization in its fully expanded form. Letting $M^{a,b,\dots,z}$ denote $M^a M^b \dots M^z$, the TF expansion for $d = 4$ is given by

$$\begin{aligned} A &= D^4 + L^4 (D^3 + L^3 (D^2 + L^2 (D^1 + L^1 D^0 R^1) R^2) R^3) R^4 \\ &= D^4 + L^4 D^3 R^4 + L^{4,3} D^2 R^{3,4} + L^{4,3,2} D^1 R^{2,3,4} + L^{4,3,2,1} D^0 R^{1,2,3,4} \end{aligned}$$

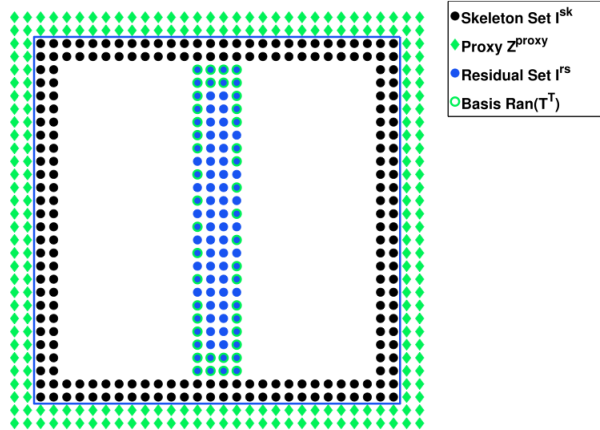
The TF is stored on a binary tree that is constructed by recursively subdividing the domain, where a box (node in the tree) is bisected into its two children. We define the index sets for box i :

I_i^{smpl}	-	sample set	-	all the sample points inside box i
I_i^{src}	-	source set	-	re-blocked degrees of freedom $I_{c_1(i)}^{sk} \cup I_{c_2(i)}^{sk}$
I_i^{sk}	-	skeleton set	-	subset of I_i^{src} , handpicked or found by ID
I_i^{rs}	-	residual set	-	non-skeleton points $I_i^{src} \setminus I_i^{skel}$

Starting at the leaf boxes with $I^{src} = I^{smp\ell}$, the computation of the TF is performed in an upward pass of the tree. Given I_i^{src} for each box at level ℓ in the tree, we compute I_i^{sk} and matrices L_i , D_i , R_i . Upon a reordering of indices, these matrices make up the diagonal components of L^ℓ , D^ℓ , R^ℓ in the TF. We then move up to level $\ell - 1$, taking the source sets to be the union of the children skeletons.

Section 4.1 of the paper shows that we achieve linear complexity of an algorithm if the per-box operations have cost and storage $o(m)$, where m is the size of the sample set. The HSS compression procedure requires us to construct skeleton sets and find L_i and R_i for each box. For HSS1D, we merge skeleton sets of children and perform an interpolative decomposition using equivalent density acceleration (proxy points). Performing an ID is prohibitively expensive for HSS2D because the interaction rank, or skeleton size growth, is $\mathcal{O}(m^{1/2})$. Instead, for a given box the skeleton set is chosen a priori to consist the points along the boundary. Consequently, the residual set consists the points along the interface of the children. Looking at the figure below, it is clear that the matrix of interactions between the proxy and skeleton can be compressed as HSS1D, and the matrix between the proxy and residual is low rank. Thus, each interpolation matrix can be computed and stored efficiently.

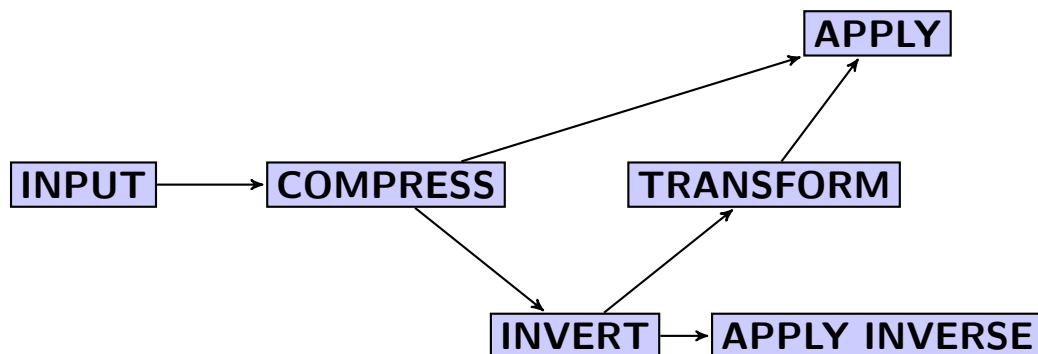
General HSS inversion is outlined in Algorithm 1 of the paper. The analogous $\tilde{L}$, $\tilde{D}$, $\tilde{R}$ matrices for the inverse can be represented in terms of the interpolation operators L_i and R_i and the auxiliary matrices E_i and F_i^{-1} . So the procedure boils down to computing E_i and F_i^{-1} at every box. For HSS1D we compute/store them densely. To achieve linear complexity in HSS2D, we compress E_i as HSS1D and we compress blocks of F_i^{-1} as low-rank or HSS1D. This is outlined in Algorithm 2 and detailed in Algorithms 3,4,5. The algorithm for HSS matrix-vector multiplication is described in Section 2.3 of the paper. For HSS2D, we use the same method but with fast HSS1D and low-rank operations. The details for the HSS2D inverse apply is outlined in Algorithm 6.



3 HSS1D - Integral Equations on Planar Curves

3.1 Dependency List

The dependencies between the different stages of the implementation are shown in the tree below.



INPUT	: input kernel, domain, parameters, and discretization (Kernel_Eval.m, HSS_tree_parameters.m)
COMPRESS	: build the binary tree and compute L, D, R of the TF. (HSS1D_compress_*.m)
APPLY	: apply HSS1D matrix to a vector: Ax (HSS1D_apply_*.m)
INVERT	: compute Schur complement matrices E and F^{-1} (HSS1D_invert_*.m)
TRANSFORM	: transform inverse to HSS form (HSS1D_transforminv_*.m)
APPLY INVERSE	: apply HSS1D matrix inverse to a vector: $A^{-1}x$ (HSS1D_applyinv_*.m)

3.2 Test Functions

In HSS1D/Tests/ there are files that test the output of certain HSS1D functions. They also serve as good examples on how to run the code.

Test_HSS1D_compress

Tests HSS1D functions compress, invert, transforminv, apply, and applyinv. Given random vectors, the output of apply and applyinv are compared to their dense counterparts. This is a good example of all stages defined in the previous section.

Test_HSS1D_compress_offd_given_tree

Tests the 'green' and 'offd' variations of HSS1D compress with a precomputed binary tree.

Test_HSS1D_merge_and_sum

This mimics the HSS2D_build_Finv routine. Tests HSS1D functions compress, split, merge, and sum.

Test_HSS1D_split

This tests the HSS1D split function. Given a boolean index vector B , HSS1D split produces HSS forms for $K(B, B)$ and $K(!B, !B)$. The following sets of indices are tested: first half, first quarter, even, and randomly picked.

Test_HSS1D_SUM

Tests HSS1D functions sum and recompress. Given vector x and two HSS matrices A and B , the output of `apply(sum(A,B),x)` is compared to `sum(apply(A,x), apply(B,x))`.

Test_LowRank_HSS1D

Tests HSS1D function `compressLR`, which is used to compress the interpolation operators for HSS2D.

Run_HSS1D_tests

Runs the above test functions.

3.3 Setup and Input

Setup

After opening MATLAB, you need to add the path to the Common/, HSS1D/, and HSS2D/ folders. You also need to run `box_constants.m`. For example, if you are running MATLAB from the root of HSSC_DirectSolver, you could execute the following code.

```
addpath ./HSS1D
addpath ./HSS2D
addpath ./Common
warning off;
box_constants;
```

Input HSS Parameters

Kernel and solve parameters are created by calling `HSS_tree_parameters.m`. The code is set up for the integral equations explored in the paper, which include variations of Laplace, Yukawa, Helmholtz, and Lippmann-Schwinger kernels on the square domain $[-1, 1] \times [-1, 1]$. For more information see `DOCUMENTATION_*.txt` or use MATLAB `help`.

Input Kernel Function (optional)

To input your own kernel, you need to edit `Kernel_Eval.m` before running `HSS_tree_parameters`. The following code block lets you specify your kernel function. Whatever name you choose should be the first argument (`pot='Kernel_Name'`) when calling `HSS_tree_parameters`.

```
elseif strcmp(params.flag_pot, 'Kernel_Name')
    % define distance function den
    % define kernel A = (h^2)/2 b(X) K(|X-Y|) C(Y)
    % den = (X_g1-Y_g1).^2 + (X_g2-Y_g2).^2;
    % A = (h^2)/2 * b(X_g1,X_g2) K(den) c(Y_g1,Y_g2);
end
```

Input Discretization

You need to define an array of points along your curve, which will be used as input for other HSS1D functions. For example, the following code generates 1000 points along the unit circle.

```
N = 1000;
h = 2*pi/N;
X = [cos(h:h:2*pi)', sin(h:h:2*pi)'] ;
```

3.4 Sample Run

In the following code, the kernel is an unsymmetric variation of the 2D Laplace kernel, the domain is the unit circle, and the tree is the default uniform binary tree. The input parameters Ng, np, and lay have no effect on HSS1D functions.

```
N = 1000; % DISCRETIZATION
h = 2*pi/N;
X = [cos(h:h:2*pi)', sin(h:h:2*pi)'] ; % 1000 points along circle
x = randn(N,1); % vector in HSS mat-vec operations
tree = 50; % integer or fully specified binary tree

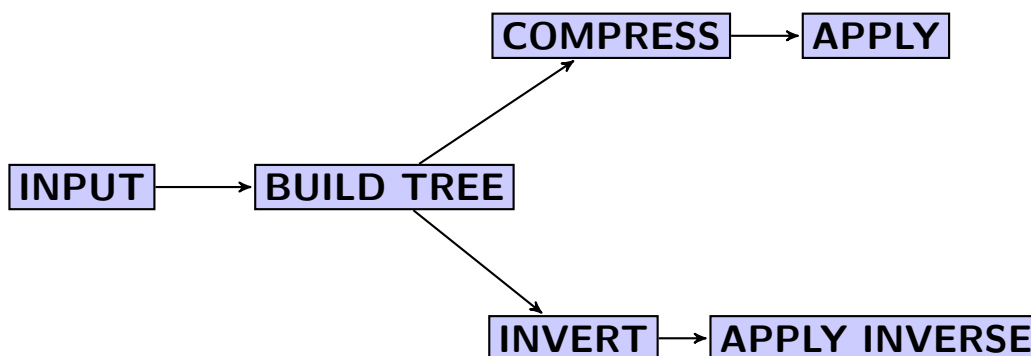
flag = 'SL_L_2D'; kh = 0; sym = 0; Ng = 1; np = 1; % INPUT
acc = 1e-10; lay = 1; n_cut = 432; TI = 0;
params = HSS_tree_parameters (flag,kh,sym,Ng,np,acc,lay,n_cut,TI);

A_HSS = HSS1D_compress_nsym ('green',X,tree,acc,params); % COMPRESS
y = HSS1D_apply_nsym (A_HSS,x); % APPLY
[HSSinv,FTinv] = HSS1D_invert_nsym (A_HSS,acc); % INVERT
Ainv_HSS = HSS1D_transforminv_nsym(A_HSS,HSSinv,FTinv); % TRANSFORM
yi = HSS1D_applyinv_nsym (A_HSS,HSSinv,FTinv,x); % APPLY INVERSE
yi2 = HSS1D_apply_nsym(Ainv_HSS,x); % APPLY (hss inverse)
```

4 HSS2D - Integral Equations on Planar Surfaces

4.1 Dependency List

The dependencies between the different stages of the implementation are shown in the tree below. Unlike HSS1D, most of the compression is done in the **BUILD TREE** stage. Thus, **COMPRESS** and **CONSTRUCT INVERSE** are independent; you need not finish the HSS compression of A to construct the compressed form of A^{-1} .



INPUT	: input kernel, domain, and solve parameters (Kernel_Eval.m, HSS_tree_parameters.m)
BUILD TREE	: tree, index sets, interpolation operators L, R (HSS2D_bintree_*.m, levHSS.m)
COMPRESS	: compute self/sibling interaction matrices D (HSS2D_compress_*.m)
APPLY	: apply HSS2D matrix to a vector: Ax (HSS2D_apply_*.m)
INVERT	: compute Schur complement matrices E and F^{-1} (HSS2D_invert_*.m)
APPLY INVERSE	: apply HSS2D matrix inverse to a vector: $A^{-1}x$ (HSS2D_applyinv_*.m)

4.2 Test Functions

In HSS2D/Tests/ there are files that test the output of certain HSS2D functions. They also serve as good examples on how to run the code.

Test_HSS2D_bintree and Test_HSS2D_skeletons

Test the output of HSS2D_build_tree_*, which includes the binary tree along with interpolation operators.

Test_Matrix_Apply

Tests the output of HSS2D_compress_* and HSS2D_apply_*, that is, computes the error of the HSS2D compressed matrix and the error of applying it to a vector. It provides a good example on how to run the compress and matrix apply routines.

Test_InverseCompression

Tests the output of `HSS2D_invert_*` and `HSS2D_applyinv_*`, that is, computes the error of the HSS2D inverse and the error of applying it to a vector. It provides a good example on how to run the invert and inverse apply routines (solve the system).

`Run_HSS2D_tests`

Runs the above test functions.

`Test_HSS2D_CMZ2012_paper*`

Tests the error of solving the examples in Section 5 of the paper. It runs `Test_HSS2D_InverseCompression` on Laplace 2D, Helmholtz 2D, and Lippman-Schwinger 2D kernels.

4.3 Setup and Input

See section 3.3 for details about setup, solve parameters, and kernel functions.

Input Discretization (optional)

The default discretization is a regular grid of $[-1, 1]^2$. To define your own discretization X , change `params.X_source` to X after running `HSS_tree_parameters`. Note that for the $\mathcal{O}(n)$ solve, X cannot be too irregular. The preset skeleton sets require each box in the quadtree to have sample points along its boundary. This will be more robust in later releases of the code.

```
X = % user defined
params = HSS_tree_parameters(..)
params.X_source = X
```

4.4 Sample Run

In the following code, the kernel is the 2D Laplace kernel and the discretization is a 128×128 grid of $[-1, 1]^2$. The two variables `params.lev_HSS` and `params.Fopts` are used to specify how auxiliary matrices E and F^{-1} are stored (see help `HSS2D_invert_*`).

```
flag_pot = 'SL_L_2D'; kh = 0; sym = 1; Ng = 16; np = 8; % INPUT
acc = 1e-10; lay = 2; n_cut = 400; TI = 1;
params = HSS_tree_parameters(flag_pot, kh, sym, Ng, np, acc, lay, n_cut, TI);

x = randn(Ng*Ng*np*np, 1);
A_HSSf = HSS2D_build_tree_fsym(params); % BUILD TREE
params.lev_HSS = lev_HSS(A_HSSf, params.n_cut, params.transinv);
params.Fopts = 'HSS';
A_HSSn = HSS2D_compress_fsym(A_HSSf, params); % COMPRESS
y = HSS2D_apply_fsym(A_HSSf, A_HSSn, x, params); % APPLY
Ai_HSS = HSS2D_invert_fsym(A_HSSf, params); % INVERT
yi = HSS2D_applyinv_fsym(A_HSSf, Ai_HSS, x, params); % APPLY INVERSE
```